

TECHNICAL REFERENCE GUIDE

Qwik Guide: Applying Machine Learning

Bias & Variance · Cross-Validation · Metrics · Model Selection · AWS ML

A beginner-friendly companion for the AI Foundations Series.

Plain-language definitions, worked processes, and diagrams — a partner volume to the CNNs & RNNs and Unsupervised Learning guides.

Building a model is only half the job. The harder half is knowing whether it's any good, and picking the best one from many candidates. This guide covers the two sources of model error, how to test a model honestly, the metrics that grade classification and regression, and how to run the whole workflow on AWS — defining every term as it appears.

AI Foundations Series · Machine Learning Practice

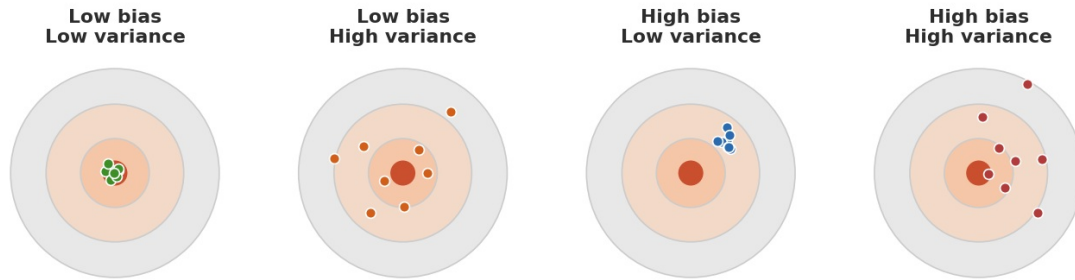
The Two Kinds of Error

Every model makes mistakes, and those mistakes come in two flavors. Understanding them is the foundation of applying machine learning well: almost every choice you make — which model, how complex, how much data — is really a choice about how to balance these two.

TIP The single biggest idea in this guide: you can't drive both errors to zero at once. Lowering one usually raises the other. Good modeling is finding the balance point, not chasing perfection.

1 Bias and Variance

- **Bias** — error from *making assumptions*. A high-bias model is too simple to capture the real pattern, so it misses relationships in the data. Examples: linear regression, Naive Bayes.
- **Variance** — error from *fluctuations in the data* (noise). A high-variance model is so flexible it memorizes the noise, so training it on different data gives very different results — a sign of **overfitting**.



Bias is how far the shots sit from the bullseye; variance is how scattered they are. The goal is the leftmost target — low bias and low variance.

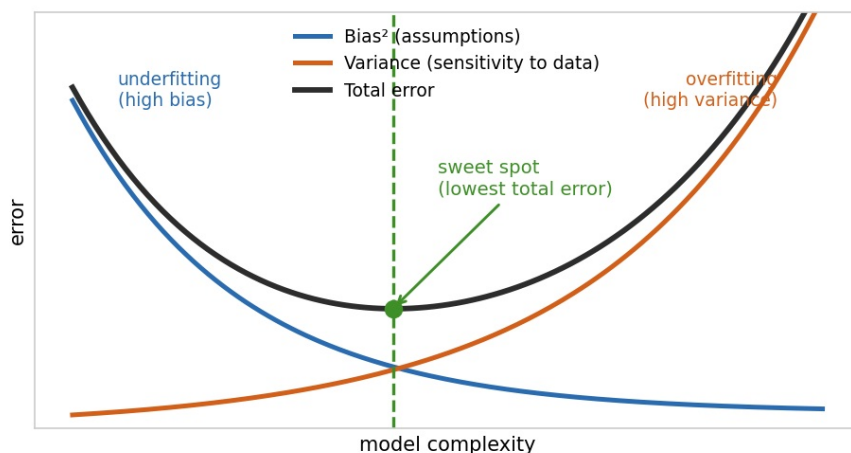
The tradeoff

Every model carries some bias and some variance. As a rule of thumb, **lowering one often raises the other**, so you're usually looking for a balance rather than driving both to zero. Tuning a model's parameters, adding **regularization**, or combining models into an **ensemble** can improve that balance.

WHICH MODELS LEAN WHICH WAY **High bias:** linear regression with regularization, k-nearest neighbors with a high k, a shallow or pruned decision tree, a neural network with few hidden units. **High variance:** plain linear regression, k-NN with a low k, a deep decision tree, a neural network with many hidden units.

2 The Tradeoff, Drawn

As a model grows more complex, bias falls but variance climbs. Total error is the sum of the two, so it dips to a low point and then rises again. That low point is the model you want.



Too simple (left) underfits with high bias; too complex (right) overfits with high variance. The sweet spot minimizes total error.

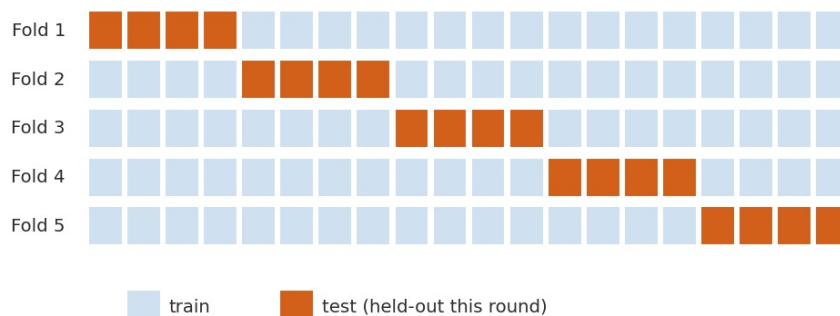
3 Cross-Validation

To know how well a model **generalizes** to new data, you test it on data it didn't train on. The usual split is three sets:

- **Training set** — the data the model learns from.
- **Test set** — held back to check performance on unseen examples. (Testing on the training set invites overfitting — the model can just memorize the answers.)
- **Validation set** — an extra held-out set used to tune the model without contaminating the test set.

But splitting a dataset into three can leave each piece too small to train well. **Cross-validation** solves this: split the data into k equal folds, then train and score k times, each time holding out a different fold as the test set. Average the scores. It costs more computation but gives a much more reliable estimate.

5-fold cross-validation: every slice is the test set exactly once



In 5-fold cross-validation the data is split five ways; each fold serves as the test set exactly once, and the five scores are averaged.

TIP Keep a separate final test set, use cross-validation only on the training data, and don't let information leak

4 Metrics for Classification

Once a classifier makes predictions, you need to grade them. The starting point is the **confusion matrix**: a table that maps every prediction (yes/no) against the true answer (yes/no), giving four outcomes. This binary version is the foundation — problems with more than two classes use *extensions* of the same idea (covered shortly).

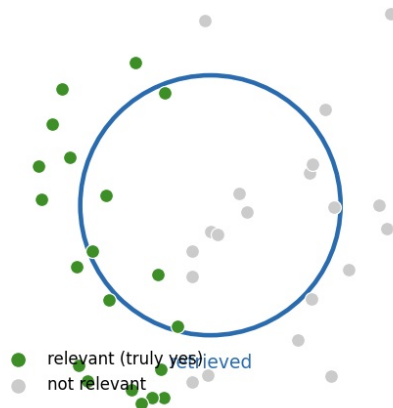
		Predicted	
		Yes	No
Actual	Yes	True Positive correct: predicted yes, actually yes	False Positive wrong: predicted yes, actually no
	No	False Negative wrong: predicted no, actually yes	True Negative correct: predicted no, actually no

The four outcomes of a binary classifier. The two green cells are correct; the two orange cells are the two ways of being wrong.

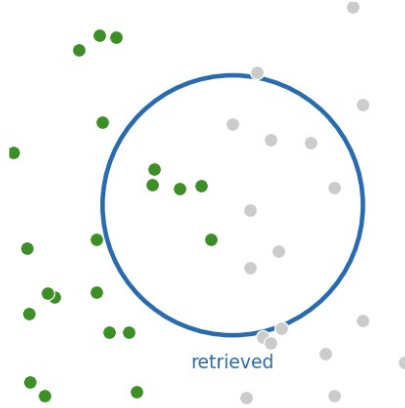
Precision and recall

- **Precision** — of everything the model flagged as yes, how much was actually yes: $TP \div (TP + FP)$.
- **Recall** — of everything that truly was yes, how much the model caught: $TP \div (TP + FN)$.

Precision = correct ÷ retrieved



Recall = correct ÷ all relevant



Precision asks how clean the retrieved set is; recall asks how much of what mattered was retrieved. Improving one often costs the other.

TIP Precision and recall pull against each other — the next page walks through why, with everyday examples.

● Spotlight: Precision vs. Recall

This is the pair beginners mix up most, so it's worth slowing down. Both measure a classifier that predicts "yes," but they answer *different questions* about the yes-es.

Precision

Of everything the model **flagged as yes**, how much was actually yes?

$$TP \div (TP + FP)$$

"When it says yes, can I trust it?"

Recall

Of everything that **truly was yes**, how much did the model catch?

$$TP \div (TP + FN)$$

"Did it find all the ones that matter?"

Everyday examples

Spam filter. High **precision** means almost nothing in your spam folder is real mail — when it flags spam, it's right. High **recall** means almost no spam sneaks into your inbox — it catches nearly all of it. **Here you lean toward precision:** losing a real email to the spam folder is worse than seeing the occasional spam.

Medical screening test. High **recall** means the test catches nearly everyone who actually has the disease — few sick people are missed. High **precision** means most positive results are truly sick. **Here you lean toward recall:** missing a real case (a false negative) is far more dangerous than a false alarm you can re-test.

Fraud detection. High **recall** catches more fraud but flags more honest transactions; high **precision** bothers fewer honest customers but lets more fraud slip through. **The right balance depends on which mistake costs more** — blocked customers or missed fraud.

WHY YOU CAN'T MAX BOTH Flagging *more* things as yes catches more real positives (recall up) but also sweeps in more wrong ones (precision down). Flagging *fewer* does the reverse. The score threshold (in the AWS section) is exactly the dial that slides between them — and F1 (next page) is one way to score the balance with a single number.

5 Combining Precision and Recall

Because precision and recall trade off, we often want a single number that balances them.

- **F1 score** — weighs precision and recall equally. $F1 = 1$ is perfect; $F1 = 0$ is worst.

$$F1 = 2 \cdot (\text{precision} \cdot \text{recall}) \div (\text{precision} + \text{recall})$$

- **F-beta / alternate F-measures** — a generalized F-score that can weight one over the other. $\text{Beta} < 1$ makes precision matter more; $\text{beta} > 1$ makes recall matter more.
- **Matthews Correlation Coefficient (MCC)** — a balanced score from -1 to 1 (1 = perfect, 0 = random, -1 = fully inverted). It uses *all four* cells of the confusion matrix (TP, FP, FN, TN), which makes it informative even when class sizes are very imbalanced.
- **Cohen Kappa Score** — compares the model against random guessing. Roughly: 0.8 – 1 strong, 0.2 – 0.6 fair/moderate, 0 – 0.2 slight, below 0 no better than random.

6 Beyond Binary: Multiclass Metrics

When there are more than two classes, the same ideas extend:

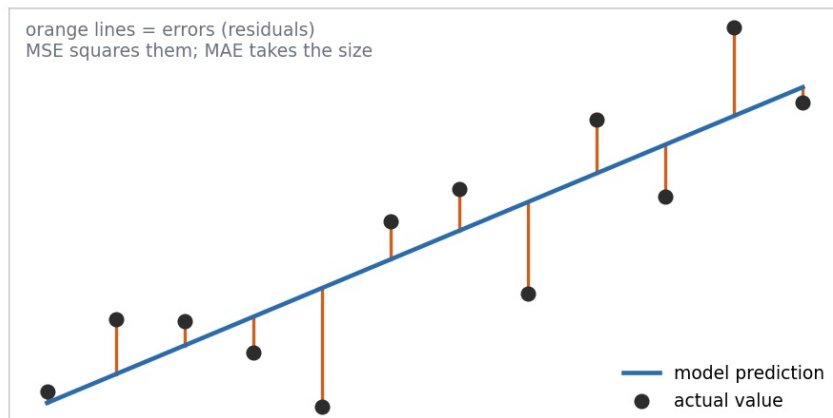
- **Multiclass metrics** — treat each class as "this class vs. everything else" and combine. A multiclass F1 is the weighted average of each class's F1.
- **Multilabel & ranked metrics** — when each input can carry several labels, or the output is a ranked list, metrics split per label or per position.
- **Average metrics** — ways to combine per-class scores: the *macro* average treats every class equally; the *weighted* average weights classes by how common they are; the *micro* average pools all the counts first, then computes the metric.

IN PYTHON scikit-learn's `sklearn.metrics` module has ready functions (`accuracy_score`, `f1_score`, and more). Some work only for binary problems; others handle multiclass and multilabel.

7 Metrics for Regression

Regression predicts a number, not a class, so its metrics measure *how far off* the predictions are. Each prediction has an error — the gap between the predicted value and the actual value (the **residual**).

Regression error: distance between prediction and actual



Regression error is the distance between each actual point and the model's line. The metrics below are different ways of summarizing those distances.

- **Mean squared error (MSE)** — the average of the squared errors; often used as a **cost function**. 0 is best; bigger means more error. Because it squares the errors, it penalizes large misses much more heavily than small ones. Commonly used — though the best choice depends on the task.
- **Mean absolute error (MAE)** — the average of the error sizes without squaring. Simpler and more interpretable, and less sensitive to large outliers — often the better pick when outliers shouldn't dominate.
- **Mean squared log error (MSLE)** — good for exponential-growth data, where the output should be treated on a log scale.
- **R² score** — how well the model explains the data. 1 is best; 0 means the model does no better than always predicting the average; below 0 means it does worse than that.

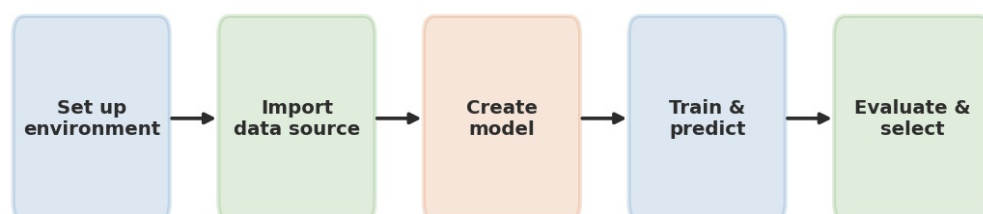
WHICH METRIC? There's no single "best" regression metric. MSE is common and punishes big errors hardest; MAE is more interpretable and robust to outliers; R² tells you how much of the variation the model explains. Pick based on whether outliers matter and what the task actually cares about.

IN PYTHON The same `sklearn.metrics` module covers regression — `mean_squared_error`, `r2_score`, and others, each with code examples in the scikit-learn docs.

8 Machine Learning on AWS

You don't have to build everything from scratch. **AWS machine learning services** are managed cloud tools that let you train models and get predictions without implementing algorithms yourself. They lower the barrier a lot — but you still need a working grasp of the basics (choosing a model, picking features, setting thresholds, reading the metrics) to use them well. Most follow a pay-per-use model: you're billed for what you consume, with no large upfront commitment.

A NOTE ON NAMING The original standalone "Amazon Machine Learning" product has been retired; today the same workflow is spread across specific services (for example Amazon SageMaker for building and training). The *steps* below are what matters and carry over regardless of which service you use.



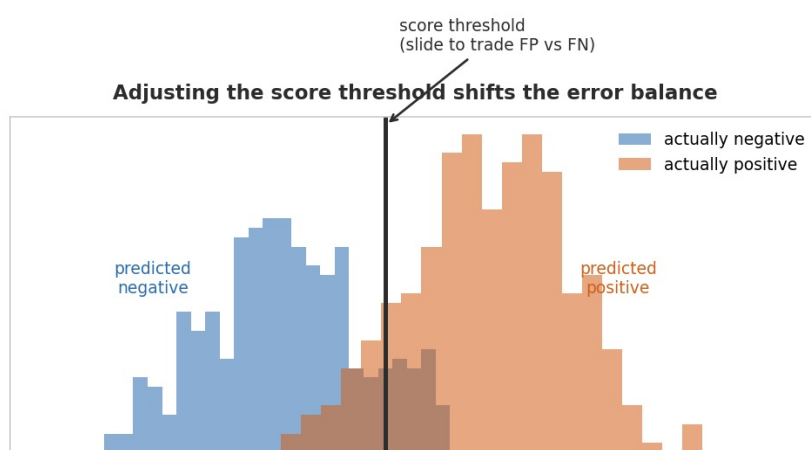
The AWS ML workflow, end to end: set up, import data, create a model, train and predict, then evaluate and select.

Key concepts

- **Data sources** — the datasets or systems your input data comes from (files, databases, storage buckets).
- **ML models** — the models you set up that generate predictions from patterns in your data.
- **Evaluations** — measures of how good a model is.
- **Batch vs. real-time predictions** — batch scores a whole file of inputs at once; real-time predictions respond per observation via an endpoint.
- **Access** — through the web console, the AWS CLI, the API, or SDKs for integrating with software.

Adjusting the score threshold

After training a binary classifier, you can slide a **score threshold** — the cutoff above which a prediction counts as "positive." Moving it changes the balance of true positives, true negatives, false positives, and false negatives. There's no universal best position: the right threshold depends on how common each class is and on the **relative cost of a false positive versus a false negative** for your problem.



★ Concept Check

The points students most often mix up on this material. Read each as "the question, then the answer."

What are the two kinds of model error?

Bias (from assumptions — too simple) and **variance** (from noise — too flexible).

Precision vs. recall?

Precision = correct ÷ everything flagged. **Recall** = correct ÷ everything that truly was positive.

What is the bias-variance tradeoff?

As a rule of thumb, lowering one often raises the other — so you **balance** them rather than eliminate both. Regularization and ensembles help.

What is the F1 score?

A single number balancing precision and recall equally. **1 is best, 0 is worst.**

High bias usually means...?

Underfitting — the model is too simple. High variance usually means **overfitting**.

What makes MCC "balanced"?

It uses **all four confusion-matrix cells**, so it stays informative even when class sizes are very imbalanced; it ranges from -1 to 1 .

Why use cross-validation instead of one split?

It reuses the data by testing on every fold once, giving a **stronger, more reliable** estimate when data is limited.

MSE vs. MAE — how do they differ?

MSE squares errors, so big misses hurt most; **MAE** uses error size, so it's more robust to outliers. Neither is always best.

Which set do you cross-validate on?

The **training data only** — keep a separate final test set, and don't leak between folds.

What does an R^2 of 0 mean?

The model is **no better than always predicting the average**. Below 0 is worse than that; 1 is best.

What does the confusion matrix show?

The four outcomes: **true positive, false positive, false negative, true negative**.

Batch vs. real-time predictions in AWS?

Batch scores a whole file at once; **real-time** answers per observation through an endpoint.

★ Quick Reference

Concept	Type	Remember this
Bias	Error	Error from assumptions; high bias = too simple = underfitting
Variance	Error	Error from noise; high variance = too flexible = overfitting
Bias-variance tradeoff	Error	Lowering one raises the other; find the balance point
Overfitting	Error	Model memorizes noise; great on training data, poor on new data
Training / test / validation sets	Validation	Learn on train, tune on validation, judge on the held-out test set
Cross-validation	Validation	Split into k folds; test on each once; average the scores
Confusion matrix	Classification	TP, FP, FN, TN — predictions vs. actual answers
Precision	Classification	$TP \div (TP + FP)$ — how clean the positives are
Recall	Classification	$TP \div (TP + FN)$ — how many real positives were caught
F1 score	Classification	Balances precision & recall equally; 1 best, 0 worst
MCC / Cohen Kappa	Classification	Balanced scores; work with imbalanced classes or vs. random
Macro / micro / weighted average	Multiclass	Ways to combine per-class scores into one number
MSE	Regression	Average squared error; punishes big misses hardest; 0 is best
MAE	Regression	Average error size; more interpretable, robust to outliers
R² score	Regression	How well the model explains the data; 1 best, 0 = average, <0 worse
AWS ML workflow	AWS	Set up → import data → create → train/predict → evaluate
Score threshold	AWS	Slide it to trade false positives against false negatives

Classification vs. regression metrics — at a glance

	Classification	Regression
Predicts	A class (yes/no, category)	A number
Built on	The confusion matrix (TP/FP/FN/TN)	The residual (prediction – actual)
Key metrics	Precision, recall, F1, MCC, Cohen Kappa	MSE, MAE, MSLE, R²
Best value	F1 = 1, MCC = 1	MSE = 0, R² = 1

★ Further Resources

Each section runs from conceptual introductions to hands-on implementation.

Bias and Variance

- **IBM — What Is the Bias-Variance Tradeoff?** — beginner overview with diagrams and examples.
<https://www.ibm.com/think/topics/bias-variance-tradeoff>
- **Google — Generalization and Overfitting** — why models do well in training but poorly on new data.
<https://developers.google.com/machine-learning/crash-course/overfitting/overfitting>
- **Scikit-learn — Underfitting vs. Overfitting** — visual, coded comparison of high-bias, balanced, and high-variance models.
https://scikit-learn.org/stable/auto_examples/model_selection/plot_underfitting_overfitting.html
- **Stanford — The Bias-Variance Tradeoff** — concise conceptual reading using classifiers.
<https://nlp.stanford.edu/IR-book/html/htmledition/the-bias-variance-tradeoff-1.html>
- **Stanford CS229 — Bias/Variance** — video lecture and notes for a more mathematical treatment.
<https://see.stanford.edu/Course/CS229/53>

BEGINNER TAKEAWAY High bias → underfitting; high variance → overfitting; the goal is a model that **generalizes** to unseen data. Suggested order: IBM → Google → Scikit-learn → Stanford.

Cross-Validation

- **Google ML Glossary — Cross-Validation** — short definition of cross-validation and k-fold.
<https://developers.google.com/machine-learning/glossary#cross-validation>
- **Google — Dividing a Dataset** — introduces training, validation, and test sets.
<https://developers.google.com/machine-learning/crash-course/overfitting/dividing-datasets>
- **IBM — Model Selection** — k-fold within the broader model-selection process.
<https://www.ibm.com/think/topics/model-selection>
- **Scikit-learn — Visualizing Cross-Validation** — K-Fold, Stratified K-Fold, and Group K-Fold compared.
https://scikit-learn.org/stable/auto_examples/model_selection/plot_cv_indices.html
- **Scikit-learn — Cross-Validation Guide** — Python examples with `cross_val_score` and `cross_validate`.
https://scikit-learn.org/stable/modules/cross_validation.html

BEGINNER TAKEAWAY Keep a separate final test set, use cross-validation only on the training data, and prevent information from leaking between folds.